baccarat

Miscellaneous (misc)

Starting bankroll: 1000 Target bankroll: 100000

Key details from server.py: Tables are drawn from a fixed roster of matchups (weighted), e.g. OmniCybr vs BlackShard, NorthStar vs VoltaicAI, etc. The only useful pre-bet info is the two AI names. Correctly identifying the stronger side, then exploiting it over the table's 12 rounds, is the intended path. Agent comments make the strength order obvious: BlackShard is intentionally weak (timid draw policy). Stronger brands (NorthStar, OmniCybr, NipCat) play closer to a sensible tableau; VoltaicAI sits in the middle.

Finding the edge

`tournament_stats.py` simulates 10k resolved rounds per pairing. Relevant roster matchups:

PlayerAI	BankerAI	Favored side	Approx. winrate
OmniCybr	BlackShard	player	~57.7%
BlackShard	OmniCybr	banker	~56.0%
NorthStar	BlackShard	player	~57.1%
BlackShard	NorthStar	banker	~57.4%
NipCat	BlackShard	player	~56.2%
BlackShard	NipCat	banker	~55.5%
VoltaicAI	BlackShard	player	~54.9%
BlackShard	VoltaicAI	banker	~52.5%
OmniCybr	VoltaicAI	player	~52.8%
VoltaicAI	OmniCybr	banker	~52.3%

PlayerAI	BankerAI	Favored side	Approx. winrate
NorthStar	VoltaicAI	player	~52.1%
VoltaicAI	NorthStar	banker	~52.6%

Rule of thumb: **bet against BlackShard**; otherwise bet the stronger brand (NorthStar / OmniCybr over VoltaicAI).

Exploit:

```
#!/usr/bin/env python3
"""Solve the baccarat duel betting challenge via pwntools.

Each table shows PlayerAI / BankerAI names. Bet the statistically
favored
side and size with fractional Kelly until the bankroll target is
hit.
"""

from __future__ import annotations

import math
import re
import sys

from pwn import context, log, remote

HOST = "baccarat-d068f8a62f6c.inst.omnictf.com"
PORT = 1337
TARGET = 100_000
KELLY_FRACTION = 0.45  # half-ish Kelly; safer against variance
MIN_BET = 1

# Resolved win probability for the favored side, from
tournament_stats.py
# (10k resolved rounds per pairing). Keys are (PlayerAI, BankerAI).
MATCHUPS: dict[tuple[str, str], tuple[str, float]] = {
    ("OmniCybr", "BlackShard"): ("player", 0.577),
    ("BlackShard", "OmniCybr"): ("banker", 0.560),
    ("NorthStar", "BlackShard"): ("player", 0.571),
```

```
    ("BlackShard", "NorthStar"): ("banker", 0.574),
    ("NipCat", "BlackShard"): ("player", 0.562),
    ("BlackShard", "NipCat"): ("banker", 0.555),
    ("VoltaicAI", "BlackShard"): ("player", 0.549),
    ("BlackShard", "VoltaicAI"): ("banker", 0.525),
    ("OmniCybr", "VoltaicAI"): ("player", 0.528),
    ("VoltaicAI", "OmniCybr"): ("banker", 0.523),
    ("NorthStar", "VoltaicAI"): ("player", 0.521),
    ("VoltaicAI", "NorthStar"): ("banker", 0.526),
}

# Fallback ranking if an unexpected pairing appears.
STRENGTH = {
    "NorthStar": 5,
    "OmniCybr": 4,
    "NipCat": 3,
    "VoltaicAI": 2,
    "BlackShard": 1,
}


def favored_side(player: str, banker: str) -> tuple[str, float]:
    if (player, banker) in MATCHUPS:
        return MATCHUPS[(player, banker)]
    p_s = STRENGTH.get(player, 0)
    b_s = STRENGTH.get(banker, 0)
    if p_s >= b_s:
        return "player", 0.52
    return "banker", 0.52


def kelly_amount(bankroll: int, p_win: float) -> int:
    """Even-money Kelly: f* = 2p - 1, then take a fraction of
that."""
    edge = 2.0 * p_win - 1.0
    if edge <= 0:
        return MIN_BET
    fraction = edge * KELLY_FRACTION
    # Cap so a cold streak near target cannot wipe the whole roll.
    fraction = min(fraction, 0.25)
    amount = max(MIN_BET, int(math.floor(bankroll * fraction)))
    # Don't overshoot the target more than needed.
```

```
    need = TARGET - bankroll
    if need > 0:
        amount = min(amount, need, bankroll)
    return max(MIN_BET, min(amount, bankroll))


def parse_names(blob: str) -> tuple[str, str, int] | None:
    banker_m = re.search(r"BankerAI\s*::\s*(\S+)", blob)
    player_m = re.search(r"PlayerAI\s*::\s*(\S+)", blob)
    bank_m = re.search(r"Bankroll\s*::\s*(\d+)", blob)
    if not (banker_m and player_m and bank_m):
        return None
    return player_m.group(1), banker_m.group(1),
int(bank_m.group(1))


def main() -> int:
    context.log_level = "info"
    host = sys.argv[1] if len(sys.argv) > 1 else HOST
    port = int(sys.argv[2]) if len(sys.argv) > 2 else PORT

    io = remote(host, port, ssl=True)
    try:
        # Banner ends before the first table header.
        io.recvuntil(b"Type 'quit'")
        io.recvline()

        while True:
            chunk = io.recvuntil(b"Bet side [player/banker]:",
timeout=30).decode(
                "ascii", "ignore"
            )

            if "FLAG ::" in chunk:
                flag = re.search(r"FLAG\s*::\s*(\S+)", chunk)
                log.success(flag.group(0) if flag else chunk)
                print(flag.group(1) if flag else chunk)
                return 0
            if "bankroll depleted" in chunk:
                log.error("Bankroll depleted")
                return 1
```

```
            parsed = parse_names(chunk)
            if parsed is None:
                log.error("Failed to parse table header:\n%s",
chunk)
                return 1

            player, banker, bankroll = parsed
            side, p_win = favored_side(player, banker)
            amount = kelly_amount(bankroll, p_win)

            log.info(
                "P=%s B=%s bank=%d -> bet %s %d (p=%.3f)",
                player,
                banker,
                bankroll,
                side,
                amount,
                p_win,
            )

            io.sendline(side.encode())
            io.recvuntil(b"Bet amount")
            io.sendline(str(amount).encode())

            result = io.recvuntil(b"Bankroll ::",
timeout=30).decode("ascii", "ignore")
            bank_line = io.recvline().decode("ascii", "ignore")
            full = result + bank_line

            if "FLAG ::" in full:
                # Unlikely mid-stream, but handle anyway.
                pass

            m = re.search(r"Bankroll\s*::\s*(\d+)", full)
            if m:
                new_bank = int(m.group(1))
                won = "wins |" in full and (
                    (side == "player" and "Player wins" in full)
                    or (side == "banker" and "Banker wins" in full)
                )
                # Result line already printed by server; just track
bankroll.
```

```
                log.info("bankroll now %d%s", new_bank, " (+)" if
won else " (-)")
                if new_bank >= TARGET:
                    # Expect victory banner next loop / after blank
line.
                    tail = io.recvall(timeout=5).decode("ascii",
"ignore")
                    flag = re.search(r"FLAG\s*::\s*(\S+)", tail)
                    if flag:
                        log.success(flag.group(0))
                        print(flag.group(1))
                        return 0
                    log.success("Target reached")
                    print(tail)
                    return 0
    finally:
        io.close()


if __name__ == "__main__":
    raise SystemExit(main())
```

(Solved by Bogdan)